Lab 01

Python Basics for Artificial Intelligence

Objective:

This lab is an introductory session on **Python**. It is a powerful object-oriented programming language, comparable to Perl, Ruby, Scheme and Java. This lab will enable the students to learn the core building blocks used in AI based approaches.

Activity Outcomes:

The expected outcome of this lab activity is students' ability to:

- Understanding IDE's
- Basic use of conditionals
- Basic use of loops
- Basic use of math libraries
- Developing a basic logic inference engine

1) Useful Concepts

Programming is simply the act of entering instructions for the computer to perform. These instructions might crunch some numbers, modify text, look up information in files, or communicate with other computers over the Internet. All programs use basic instructions as building blocks.

You can combine these building blocks to implement more intricate decisions, too. Programming is a creative task, somewhat like constructing a castle out of LEGO bricks. You start with a basic idea of what you want your castle to look like and inventory your available blocks. Then you start building. Once you've finished building your program, you can pretty up your code just like you would your castle.

Python refers to the Python programming language (with syntax rules for writing what is considered valid Python code) and the Python interpreter software that reads source code (written in the Python language) and performs its instructions. The name Python comes from the surreal British comedy group Monty Python, not from the snake. Python programmers are affectionately called Pythonistas.

In regards to setting up the python environment, if you're new to Python and focused primarily on learning the language rather than building professional software, then you should install from the Microsoft Store package. This offers the shortest and easiest path to getting started with minimal hassle.

On the other hand, if you're an experienced developer looking to develop professional software in a Windows environment, then the official Python.org installer is the right choice. Your installation won't be limited by Microsoft Store policies, and you can control where the executable is installed and even add Python to PATH if necessary.

In Python,a namespace is a collection of names and the details of the objects referenced by the names. We can consider a namespace as a python dictionary which maps object names to objects. The keys of the dictionary correspond to the names and the values correspond to the objects in python. In python, there are four types of namespaces, namely built-in namespaces,global namespaces, local namespaces and enclosing namespaces.

Most recent releases of the python build can be downloaded from:

https://www.python.org/downloads/windows/

or use spyder
https://www.spyder-ide.org/download/

2) Solved Lab Activites (Allocated Time 1 Hour)

Activity 1:

Display numbers on screen using Python IDLE.

Solution:

1. Run Python IDLE
2. Type in any number, say **24** and press **Enter**.
3. **24** should be printed on the screen
4. Now type **4.2**, press **Enter**.
5. **4.2** should be displayed on screen as an output.
6. Now type **print(234)**. Press **Enter**. **234** will be the output.
7. Type **print(45.90)** and press **Enter**. The output will show **45.90** on screen.

```
1
2   print("24")
3
```

Output

```
24
```

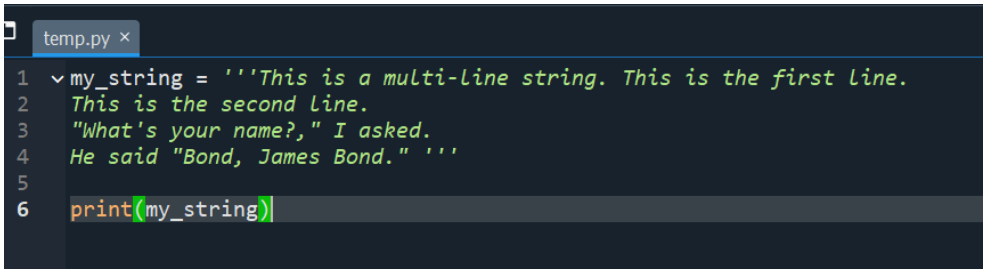
Activity 2:

Display strings on screen.

Solution:

1. Python recognized strings through quotes (single, double). Anything inside quotes is a string for Python interpreter.
2. Type **hello**, press **Enter**. An error message will be displayed as Python interpreter does not understand this as a string.
3. Type **'hello'** and press **Enter**. Hello will be displayed.
4. Type **'Quote me on this!'** and press **Enter**. Same string will be displayed.
5. Type **"What's your name?"** and press **Enter**. **What's your name?** will be printed on the screen.
6. You can specify multi-line strings using triple quotes – " " " or ' ' '. Type following text and press Enter

```
'''This is a multi-line string. This is the first line.
This is the second line.
"What's your name?," I asked.
He said "Bond, James Bond." '''
```



Output

```
This is a multi-line string. This is the first line.
This is the second line.
"What's your name?," I asked.
He said "Bond, James Bond."
```

Activity 3:

Use Python as a calculator.

Solution:

1. The interpreter acts as a simple calculator: you can type an expression at it and it will write the value. Expression syntax is straightforward: the operators + (addition), - (subtraction), * (multiplication) and / (division) work just like in most other languages.Parentheses (()) can be used for grouping.
2. Type in **2 + 2** and press **Enter**. Python will output its result i.e. **4**.
3. Try following expressions in the shell.

a. 50 – 4

b. 23.5 – 2.0

c. 23 – 18.5

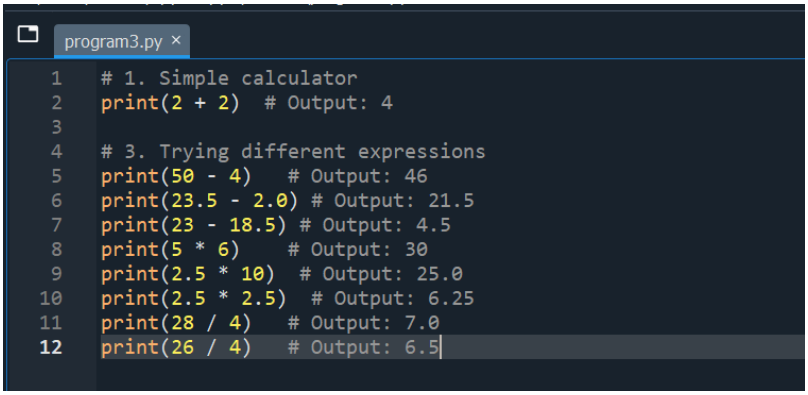
d. 5 * 6

e. 2.5 * 10

f. 2.5 * 2.5

g. 28 / 4 (note: division returns a floating point number)

h. 26 / 4



Output

Activity 4:

Get an integer answer from division operation. Also get remainder of a division operation in the output.

Solution:

2. Division (/) always gives a float answer. There are different ways to get a whole as an answer.
3. Use // operator:
    a. Type **28 // 4** and press **Enter**. The answer is a whole number.
    b. Type **26 // 4** and press **Enter**.

4. Use (int) cast operator: this operator changes the interchangeable types.

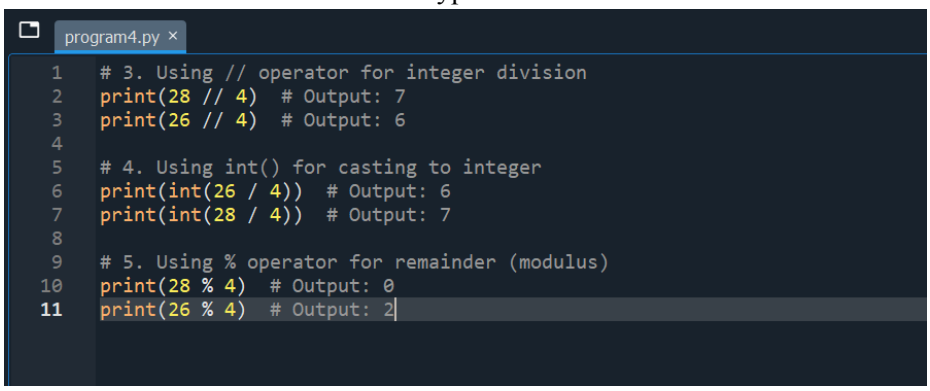
    a. Type **(int)26 / 4** and press **Enter**. The answer is a whole number.

    b. Type **(int) 28/4**; press **Enter**.

5. The modulus (or mod) % operator is used to get the remainder as an output (division / operator returns the quotient).

    a. Type **28 % 4**. Press **Enter**. **0** will be the result.

    b. Type **26 % 4**. Press **Enter**. **2** will be the result.



Output

Activity 5:

Calculate 4^3, 4^10, 4^29, 4^150, 4^1000

Solution:

1. The multiplication (*) operator can be used for calculating powers of a number. However, if the power is big, the task will be tedious. For calculating powers of a number, Python uses ** operator.
2. Type following and obtain the results of above expressions.

a. 4 ** 3

b. 4 ** 10

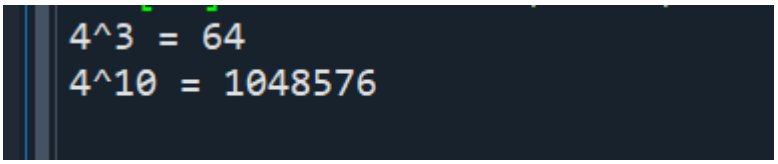
c. 4 ** 29

d. 4 ** 150

e. 4 ** 1000

```
base = 4

# Calculate powers using the ** operator
power_3 = base ** 3
power_10 = base ** 10
power_29 = base ** 29
power_150 = base ** 150
power_1000 = base ** 1000

# Print the results
print("4^3 =", power_3)
print("4^10 =", power_10)
```

Output



Activity 6: Write following math expressions. Solve them by hand using operators' precedence. Calculate their answers using Python. Match the results.

Solution:

The table below shows the operator precedence in Python (from Highest to Lowest).

Operator	Operation	Example
**	Exponent	2 ** 3 = 8
%	Modulus/remainder	22 % 8 = 6
//	Integer division/floored quotient	22//8 = 2
/	Division	22/8 = 2.75
*	Multiplication	3*5 = 15
-	Subtraction	5 – 2 = 3
+	Addition	2 + 2 = 4

Calculate following expressions:

1. 2+3*6
2. (2+3)*6
3. 48565878 * 578453
4. 2 + 2 (note the spaces after +)
5. (5 - 1) * ((7 + 1) / (3 - 1))
6. 5 +
7. 42 + 5 + * 2

```python
# 1. 2 + 3 * 6
result1 = 2 + 3 * 6
print("2 + 3 * 6 =", result1)  # Output: 20

# 2. (2 + 3) * 6
result2 = (2 + 3) * 6
print("(2 + 3) * 6 =", result2)  # Output: 30

# 3. 48565878 * 578453
result3 = 48565878 * 578453
print("48565878 * 578453 =", result3)  # Output: 28086525727334

# 4. 2 + 2 (with spaces)
result4 = 2 + 2
print("2 + 2 =", result4)  # Output: 4

# 5. (5 - 1) * ((7 + 1) / (3 - 1))
result5 = (5 - 1) * ((7 + 1) / (3 - 1))
print("(5 - 1) * ((7 + 1) / (3 - 1)) =", result5)  # Output: 16.0

# 6. 5 + 42 + 5 * 2 (modified for clarity) - This was the original: 5 + 7.42
result6 = 5 + 42 + 5 * 2
print("5 + 42 + 5 * 2 =", result6)  # Output: 57
```

Output

```
2 + 3 * 6 = 20
(2 + 3) * 6 = 30
48565878 * 578453 = 28093077826734
2 + 2 = 4
(5 - 1) * ((7 + 1) / (3 - 1)) = 16.0
5 + 42 + 5 * 2 = 57
```

Activity 7:

Combine numbers and text.

Solution:

Type the following code. Run it.

```python
x = "Nancy"
print(x)
s = "My lucky number is %d, what is yours?" % 7
print(s)
s = "My lucky number is " + str(7) + ", what is yours?"
print(s)
```

Output

```
Nancy
My lucky number is 7, what is yours?
My lucky number is 7, what is yours?
```

Activity 8:

Take input from the keyboard and use it in your program.

Solution:

In Python and many other programming languages you can get user input. In Python the input() function will ask keyboard input from the user.The input function prompts text if a parameter is given. The function reads input from the keyboard, converts it to a string and removes the newline (Enter).Type and experiment with the script below.

program8.py

```
1  name = input('What is your name? ')
2  print('Hello ' + name)
3
4  job = input('What is your job? ')
5  print('Your job is ' + job)
6
7  num = input('Give me a number? ')
8  print('You said: ' + str(num))
9
```

Output

```
What is your name? AI
Hello AI
What is your job? give sloutions
Your job is give sloutions
Give me a number? 4
You said: 4
```

Activity 9:

Let us take an integer from user as input and check whether the given value is even or not.

Solution:

A. Create a new Python file from Python Shell and type the following code.

B. Run the code by pressing F5.

```
program9.py ×
1  # Get integer input from the user
2  num = int(input("Enter an integer: "))
3
4  # Check if the number is even
5  if num % 2 == 0:
6    print(num, "is even")
7  else:
8    print(num, "is odd")
```

You will get the following output.

```
In [25]: %runfile 'C:/Users/Lenovo/.spyder-p
Enter an integer: 6
6 is even

In [26]: %runfile 'C:/Users/Lenovo/.spyder-p
Enter an integer: 7
7 is odd
```

Activity 10:

Let us modify the code to take an integer from user as input and check whether the given value is even or odd. If the given value is not even then it means that it will be odd. So here we need to use if-else statement an demonstrated below.

Solution:

A. Create a new Python file from Python Shell and type the following code.

B. Run the code by pressing F5.

```
program10.py ×
1  # Get integer input from the user
2  num = int(input("Enter an integer: "))
3
4  # Check if the number is even using if-else
5  if num % 2 == 0:
6    print(num, "is even")
7  else:
8    print(num, "is odd")
```

You will get the following output.

```
Enter an integer: 1
1 is odd
```

Activity 11:

Calculate the sum of all the values between 0-10 using while loop.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

program11.py

```
# Initialize variables
total_sum = 0
current_number = 0

# Calculate sum using while loop
while current_number <= 10:
    total_sum += current_number  # Add current number to the total sum
    current_number += 1  # Move to the next number

# Print the result
print("The sum of values between 0 and 10 is:", total_sum)
```

You will get the following output.

```
The sum of values between 0 and 10 is: 55

In [29]:
```

Activity 12:

Accept 5 integer values from user and display their sum. Draw flowchart before coding in python.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

program12.py

```
# Initialize variables
count = 0
total_sum = 0

# Get 5 numbers and calculate sum using while loop
while count < 5:
    num = int(input("Enter number " + str(count + 1) + ": "))
    total_sum += num
    count += 1

# Display the sum
print("The sum of the 5 numbers is:", total_sum)
```

You will get the following output.

```
Enter number 1: 2
Enter number 2: 3
Enter number 3: 4
Enter number 4: 5
Enter number 5: 6
The sum of the 5 numbers is: 20
```

Activity 13:

Write a Python code to keep accepting integer values from user until 0 is entered. Display sum of the given values.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

program13.py

```
# Initialize the sum
total_sum = 0

# Get input until 0 is entered
num = int(input("Enter an integer (enter 0 to stop): "))
while num != 0:
    total_sum += num  # Add the number to the sum
    num = int(input("Enter an integer (enter 0 to stop): "))

# Display the sum
print("The sum of the entered numbers is:", total_sum)
```

You will get the following output.

```
In [32]: %runfile 'C:/Users/Lenovo/.spyde
Enter an integer (enter 0 to stop): 1
Enter an integer (enter 0 to stop): 2
Enter an integer (enter 0 to stop): 3
Enter an integer (enter 0 to stop): 4
Enter an integer (enter 0 to stop): 5
Enter an integer (enter 0 to stop): 0
The sum of the entered numbers is: 15
```

Activity 14:

Write a Python code to accept an integer value from user and check that whether the given value is prime number or not.

Solution:

- Create a new Python file from Python Shell and type the following code.
- Run the code by pressing F5.

program14.py

```
# Get integer input from the user
num = int(input("Enter an integer: "))

# Check if the number is prime
if num > 1:
    for i in range(2, int(num**0.5) + 1):
        if (num % i) == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a prime number")
else:
    print(num, "is not a prime number")
```

You will get the following output(s).

```
Enter an integer: 6
6 is not a prime number

In [26]:
```

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

Lab Task 1:

Write a Python code to accept marks of a student from 1-100 and display the grade according to the following formula.

Grade F if marks are less than 50 Grade E if marks are between 50 to 60 Grade D if marks are between 61 to 70 Grade C if marks are between 71 to 80 Grade B if marks are between 81 to 90
Grade A if marks are between 91 to 100

Lab Task 2:

Write a program that takes a number from user and calculate the factorial of that number.

Lab Task 3:

Fibonacci series is that when you add the previous two numbers the next number is formed.

You have to start from 0 and 1.
E.g. 0+1=1 → 1+1=2 → 1+2=3 → 2+3=5 → 3+5=8 → 5+8=13

So the series becomes
0 1 1 2 3 5 8 13 21 34 55 ……

Steps: You have to take an input number that shows how many terms to be displayed. Then use loops for displaying the Fibonacci series up to that term e.g. input no is =6 the output should be 0 1 1 2 3 5.

Lab 02
Artificial Intelligence Agents (AI Agents)

Objective:

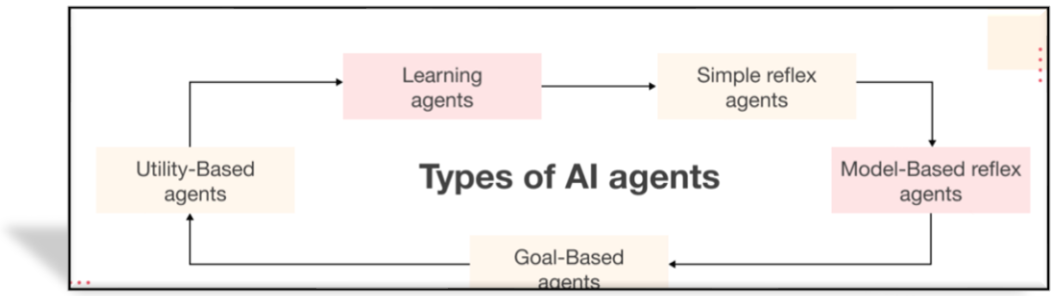
- Understand the concept of AI agents and their role in artificial intelligence.
- Identify different types of AI agents and their working principles.
- Analyze how AI agents perceive and interact with their environment.
- Implement basic AI agent logic for decision-making and problem-solving.
- Evaluate AI agent performance based on efficiency, rationality, and adaptability.

Activity Outcomes:

- Students will be able to explain the fundamental concepts and types of AI agents.
- Students will analyze how AI agents perceive and respond to their environment.
- Students will implement a basic AI agent to solve a simple problem.
- Students will evaluate the performance of AI agents based on predefined criteria.
- Students will demonstrate critical thinking in designing intelligent agent-based solution

1) Useful Concepts

- **Intelligent Agents –** Entities that perceive their environment and take actions to achieve goals.
- **Types of AI Agents –** Reflex agents, model-based agents, goal-based agents, utility-based agents, and learning agents.



- **Perception & Sensors –** How agents collect data from their environment using sensors.
- **Actuators & Actions –** Mechanisms through which agents interact with their environment.
- **Agent Environment –** Fully/partially observable, deterministic/stochastic, static/dynamic, discrete/continuous environments.

- **Rationality & Performance Measures –** How agents make decisions based on logic, goals, and efficiency.
- **Agent Architecture –** The structure of AI agents, including simple rule-based and learning-based systems.
- **Autonomous Learning** – How agents improve their performance through machine learning techniques.

2) Solved Lab activities (Allocated Time 1 Hour)

Simple Reflex agent

Activity 1:

Automatic Room Heater

```
# Automatic Room Heater(simple reflex agent)
#(React based on current percepts, without memory)
def heater_control(temperature):
    if temperature < 20:
        return "Turn ON heater"
    else:
        return "Turn OFF heater"

print(heater_control(1))  # Output: Turn ON heater


```

Output:

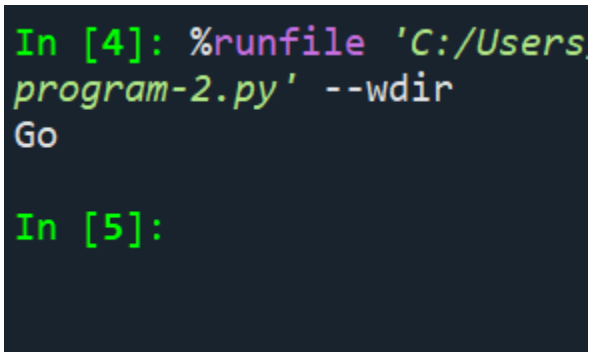
```
2/program-1.py' --wdir
Turn ON heater

In [4]:
```

Activity 2:

Traffic Light Controller

```python
#Traffic Light Controller

def traffic_light(color):
    if color == "Red":
        return "Stop"
    elif color == "Yellow":
        return "Slow Down"
    elif color == "Green":
        return "Go"
    else:
        return "Invalid Color"

print(traffic_light("Green"))  # Output: Stop

```

Output

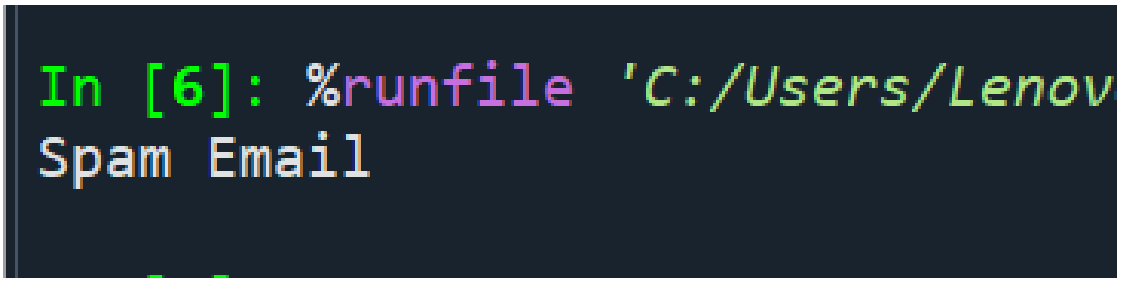


Activity 3:

Spam Email Detector

```python
#Spam Email Detector

def spam_filter(email_text):
    spam_keywords = ["lottery", "prize", "winner"]
    for word in spam_keywords:
        if word in email_text.lower():
            return "Spam Email"
    return "Not Spam"

print(spam_filter("You won a lottery!"))  # Output: Spam Email

```

Output:



Practical Questions:

- **Design AI-powered recommendation system**

Activity 4:

Model-Based Reflex Agent

Smart Vacuum Cleaner

```python
#2. Model-Based Reflex Agent-1
#Smart Vacuum Cleaner

class VacuumCleaner:
    def __init__(self):
        self.room_state = {"Living Room": "dirty", "Kitchen": "clean"}

    def clean(self, room):
        if self.room_state[room] == "dirty":
            self.room_state[room] = "clean"
            return f"Cleaning {room}"
        return f"{room} is already clean"

vacuum = VacuumCleaner()
print(vacuum.clean("Living Room"))  # Output: Cleaning Living Room
print(vacuum.clean("Kitchen"))      # Output: Kitchen is already clean

```

Output

```
program-4.py' --wdir
Cleaning Living Room
Kitchen is already clean

In [3]:
```

Activity 5:

Smart thermostat

```python
#Smart Thermostat
class Thermostat:
    def __init__(self):
        self.current_temp = 22

    def adjust_temperature(self, desired_temp):
        if self.current_temp > desired_temp:
            return "Cooling down..."
        elif self.current_temp < desired_temp:
            return "Heating up..."
        return "Temperature is perfect"

thermo = Thermostat()
print(thermo.adjust_temperature(20))  # Output: Cooling down...

```

Output

```
program-5.py' --wdir
Cooling down...
```

Activity 6:

Elevator System

```python
#Elevator System
class Elevator:
    def __init__(self):
        self.current_floor = 1

    def move_to(self, target_floor):
        if target_floor > self.current_floor:
            return "Going up"
        elif target_floor < self.current_floor:
            return "Going down"
        return "Already on the floor"

elevator = Elevator()
print(elevator.move_to(3))  # Output: Going up


```

Output:

```
Going up

In [5]:
```

Activity 7:

Goal-Based Agent

Self-Driving Car

```python
#3. Goal-Based Agent
#Self-Driving Car

def drive(destination):
    if destination == "Home":
        return "Taking shortest route to Home"
    elif destination == "Office":
        return "Taking fastest route to Office"
    return "Destination unknown"

print(drive("Home"))  # Output: Taking shortest route to Home
```

```
program-7.py' --wdir
Taking shortest route to Home
```

Activity 8:

Maze Solver

```python
#Maze Solver
def solve_maze(current_position, goal_position):
    if current_position == goal_position:
        return "Reached goal"
    return "Move towards goal"

print(solve_maze((0, 0), (3, 3)))  # Output: Move towards goa
```

Output

```
AI lab 2/program-8.py' --wdir
Move towards goal

In [8]:
```

Activity 9:

Chess game

```python
# Chess AI

def choose_move(opponent_move):
    if opponent_move == "Attack":
        return "Defend"
    return "Counter Attack"

print(choose_move("Attack"))  # Output: Defend

```

Output

```
AI lab 2/program-9.py' --wdir
Defend
```

Questions :

- **Design An AI-powered hospital scheduling system that assigns doctors to patients based on urgency and availability.**

Activity 10:

Utility-Based Agent

Shopping Cart Recommender

```python
#4. Utility-Based Agent

# Shopping Cart Recommender
def recommend_product(user_preference):
    if user_preference == "Electronics":
        return "Recommend: Smartphone"
    return "Recommend: Clothing"

print(recommend_product("Electronics"))  # Output: Recommend: Smartphone

```

Output

```
AI lab 2/program-10.py' --wd
Recommend: Smartphone
```

Activity 11:

Smart Energy Saver

```python
#Smart Energy Saver
def save_energy(usage):
    if usage > 100:
        return "Turn off non-essential devices"
    return "Energy usage is normal"

print(save_energy(120))  # Output: Turn off non-essential devices

```

Output

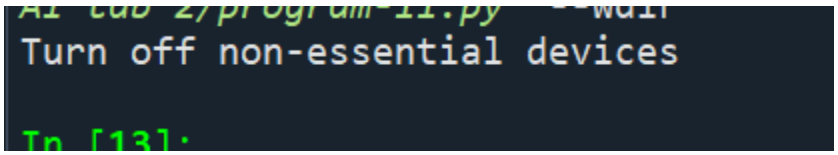
```
AI lab 2/program-10.py' --wdir
Recommend: Smartphone

In [12]:
```

Activity 12:

Autonomous Drone

```python
#Autonomous Drone
def drone_navigation(weather):
    if weather == "Clear":
        return "Proceed with flight"
    return "Delay flight"

print(drone_navigation("Rainy"))  # Output: Delay flight

```

Output



Practical questions :

- **A shopping recommendation system suggesting products based on user ratings**

Activity 13:

Learning Agent:

AI Chatbot (Using a Dictionary to Learn New Responses)

```python
# Learning Agent:
    #AI Chatbot (Using a Dictionary to Learn New Responses)

class LearningChatbot:
    def __init__(self):
        self.knowledge = {}  # Stores learned responses

    def learn(self, question, answer):
        self.knowledge[question] = answer  # Adds new knowledge

    def respond(self, question):
        return self.knowledge.get(question, "I don't know. Teach me!")  # Responds if learned

chatbot = LearningChatbot()
chatbot.learn("How are you?", "I'm good, thanks!")
print(chatbot.respond("How are you?"))  # Output: I'm good, thanks!
print(chatbot.respond("What is AI?"))   # Output: I don't know. Teach me!

```

Output

```
LAB Manuals/AI LAB1/AI lab 2
I'm good, thanks!
I don't know. Teach me!

In [14]:
```

Activity 14:

Self-Adjusting Temperature Controller (Using Basic Feedback Loop)

```python
#2. Learning Agent:
    #Self-Adjusting Temperature Controller (Using Basic Feedback Loop)

class SmartThermostat:
    def __init__(self, temp):
        self.current_temp = temp
        self.adjustments = []  # Stores past adjustments

    def adjust(self, desired_temp):
        change = desired_temp - self.current_temp  # Calculate adjustment
        self.adjustments.append(change)  # Store learning history
        self.current_temp += change // 2  # Learns by adjusting gradually
        return f"Adjusted temperature to {self.current_temp}°C"

thermo = SmartThermostat(22)
print(thermo.adjust(26))  # Output: Adjusted temperature to 24°C
print(thermo.adjust(26))  # Output: Adjusted temperature to 25°C

```

Output

```
LAB Manuals/AI LAB1/AI lab 2/pr
Adjusted temperature to 24°C
Adjusted temperature to 25°C
```

Question :

AI Chatbot(train on questions and answers according to this semester subjects)

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

1.

Create reactive agent that decides to move left or right based on its environment. It reacts only to the current state without memory or foresight.

2.

Create agent that seeks to complete tasks in a specific order (goal is to complete all tasks).

3.

An agent that makes decisions based on utility values for tasks like buying food, clothes, or electronics.

4.

Design a utility-based agent for an employee scheduling system that evaluates and chooses work shifts based on the employee's skill, availability, and shift preferences.

5.

Create a utility-based agent for a shopping assistant. The agent should maximize the total satisfaction (utility) of items bought, factoring in budget, preferences, and discounts.

6.

A logistics company faced delays in payments and high Days Sales Outstanding (DSO). By using AI to automatically create invoices, send reminders, and update payments, the company reduced its DSO by 12% and minimized manual errors. AI-driven automation can reduce DSO by 5–15% and lower manual work by over 50%. How do AI agents help a company reduce delayed payments and improve invoice management?

AI agent Extended Lab:

Simple Reflex agent

The door **opens** when motion is detected and **closes** otherwise.

```python
def automatic_door(motion_detected):
    if motion_detected:
        print("Door Opens")
    else:
        print("Door Closes")

# Testing the agent
automatic_door(True)   # Motion detected
automatic_door(False)  # No motion
```

Upgrading the Automatic Door

NewFeatures:

If motion is detected and someone is standing near → Open door

If no motion but someone is standing → Keep door open

If no motion & no one standing → Close door

2-

The traffic light turns red if a car is detected and green if no car is present.

```python
def traffic_light(car_detected):
    if car_detected:
        print("Light is RED (Car waiting)")
    else:
        print("Light is GREEN (No car)")

# Testing the agent
traffic_light(True)  # Car detected
traffic_light(False) # No car detected
```

Extended :

Enhancing Traffic Light

New Features:
If a car is detected & pedestrian is waiting → Keep red light longer
If only car detected → Normal red light
If no car & no pedestrian → Green light

3-

the street light turns on at night and off during the day.

```
def street_light(time):
    if time == "night":
        print("Street Light ON ")
    else:
        print("Street Light OFF")

# Testing the agent
street_light("night")  # Night time
street_light("day")    # Day time

```

```
reflex.py' --wdir
Street Light ON
Street Light OFF
```

extended :

smart Street Light with Weather

New Features:
If night + motion → Turn on bright light
If night + no motion → Turn on dim light
If day → Light is off
If rainy day → Turn on light (even during the day)

4- Room Temperature Controller

```python
def temperature_control(temp):
    if temp < 18:
        print("Heater ON")
    elif temp > 26:
        print("AC ON")
    else:
        print("Temperature is OK")

# Testing the agent
temperature_control(15)  # Too cold
temperature_control(30)  # Too hot
temperature_control(21)  # Normal
```

Extended:

Extended Version: Adds humidity control

5-

Dispenses sanitizer when a hand is detected.

```python
def sanitizer_dispenser(hand_detected):
    if hand_detected:
        print("Dispensing Sanitizer")
    else:
        print("Waiting for Hand...")

# Testing the agent
sanitizer_dispenser(True)  # Hand detected
sanitizer_dispenser(False) # No hand detected
```

Extended;

Checks bottle level and gives a refill alert.

6-

Car Seatbelt Alarm

Sounds alarm if seatbelt is not fastened.

```python
def seatbelt_alarm(seatbelt_fastened):
    if not seatbelt_fastened:
        print("Please Fasten Your Seatbelt!")
    else:
        print("Seatbelt Fastened.")

# Testing the agent
seatbelt_alarm(False)  # Seatbelt not fastened
seatbelt_alarm(True)   # Seatbelt fastened
```

Extended Version: Also checks for car movement before alerting.

MODEL BASED AGENT:

Automatic Fan Control

Simple Version:

- **Turns ON the fan if the temperature is too hot (>30°C).**
- **Turns OFF the fan otherwise.**

```python
class SmartThermostat:
    def __init__(self):
        self.previous_temp = None

    def update_state(self, current_temp):
        if self.previous_temp is None:
            print("Setting initial temperature...")
        elif current_temp > self.previous_temp:
            print("It's getting hotter! Turning on the AC.")
        elif current_temp < self.previous_temp:
            print("It's cooling down! Turning on the heater.")
        else:
            print("Temperature is steady.")

        # Update the previous temperature to the current one
        self.previous_temp = current_temp

# Testing
thermostat = SmartThermostat()
thermostat.update_state(22)  # Initial setup
thermostat.update_state(25)  # Getting hotter → AC ON
thermostat.update_state(18)  # Cooling down → Heater ON
thermostat.update_state(18)  # Temperature is steady
```

Extended Model-Based Version

Added a check for temperature range to decide whether the thermostat should turn off the heater or AC.

2-

Model-Based Water Tank System

Extended Version with Object Creation

```python
class WaterTank:
    def __init__(self):
        self.level = 50  # Initial water level

    def fill(self):
        self.level += 10
        return "Filling water..."

    def drain(self):
        self.level -= 10
        return "Draining water..."

    def check_level(self):
        return f"Current water level: {self.level}%"

# Creating Object and Calling Methods
tank = WaterTank()
print(tank.fill())         # Output: Filling water...
print(tank.check_level())  # Output: Current water level: 60%
print(tank.drain())        # Output: Draining water...
print(tank.check_level())  # Output: Current water level: 50%
```

Extended;

Extended Model-Based Water Tank System

Modifications:

- **Added auto-drain when full (if level > 90)**
- **Added low-water warning (if level < 20)**

3-

Model-Based Battery System

```python
class Battery:
    def __init__(self):
        self.charge = 80  # Battery starts at 80%

    def use(self):
        self.charge -= 20
        return "Using battery... Charge decreasing."

    def recharge(self):
        self.charge += 20
        return "Recharging battery..."

    def check_charge(self):
        return f"Battery charge: {self.charge}%"

# Creating Object and Calling Methods
battery = Battery()
print(battery.use())         # Output: Using battery... Charge decreasing.
print(battery.check_charge())# Output: Battery charge: 60%
print(battery.recharge())    # Output: Recharging battery...
print(battery.check_charge())# Output: Battery charge: 80%
```

Extended

xtended Model-Based Battery System

Modifications:

- **Added auto-power off when battery is critically low (charge <= 10)**
- **Added full-charge warning (charge > 100)**

Lab 04
State Space Search, Search Tree Method, (AND & OR trees)

Objective:

- Understand the concept of State Space Search in Artificial Intelligence.
- Implement an interactive 8-puzzle game.
- Develop a systematic approach to explore possible moves

Activity Outcomes:

- Students will be able to explain the fundamental concepts of State Space Search.
- Students will demonstrate critical thinking in designing intelligent agent-based solution

1) Useful Concepts

State Space Search is a problem-solving approach in AI where an algorithm explores different possible states to reach a goal. The 8-puzzle problem is a classic example of such a problem.

Activates:

Given a **2x2 sliding puzzle Game** with numbers **1 to 3** and a blank tile (**0**), the goal is to rearrange the tiles to match a predefined **goal state** by moving the blank tile in valid direction

```python
import random

# Define goal state for 2x2 puzzle
goal_state = [[1, 2], [3, 0]]

# Generate a random start state
def generate_puzzle():
    nums = [0, 1, 2, 3]
    random.shuffle(nums)
    return [nums[:2], nums[2:]]

# Find the empty tile (0) position
def find_zero(state):
    for i in range(2):
        for j in range(2):
            if state[i][j] == 0:
                return i, j

# Print the puzzle
def print_puzzle(state):
    for row in state:
        print(" ".join(str(num) if num != 0 else "_" for num in row))
    print()

# Move the empty tile
def move(state, direction):
    i, j = find_zero(state)
    new_state = [row[:] for row in state]  # Copy state

```

```
29
30      if direction == "up" and i > 0:
31          new_state[i][j], new_state[i-1][j] = new_state[i-1][j], new_state[i][j]
32      elif direction == "down" and i < 1:
33          new_state[i][j], new_state[i+1][j] = new_state[i+1][j], new_state[i][j]
34      elif direction == "left" and j > 0:
35          new_state[i][j], new_state[i][j-1] = new_state[i][j-1], new_state[i][j]
36      elif direction == "right" and j < 1:
37          new_state[i][j], new_state[i][j+1] = new_state[i][j+1], new_state[i][j]
38      else:
39          print("Invalid move!")
40      return new_state
41
42  # Main game loop
43  puzzle = generate_puzzle()
44  while puzzle != goal_state:
45      print_puzzle(puzzle)
46      move_direction = input("Move (up/down/left/right): ").strip().lower()
47      puzzle = move(puzzle, move_direction)
48
49  # Print final solved puzzle before congratulating
50  print_puzzle(puzzle)
51  print("Congratulations! You solved it!")
```

Search Tree Method

A search tree is a structured way to explore possible states or paths in a problem-solving scenario. It is commonly used in Artificial Intelligence (AI) and pathfinding problems to represent decision-making processes.

Key Components of a Search Tree:

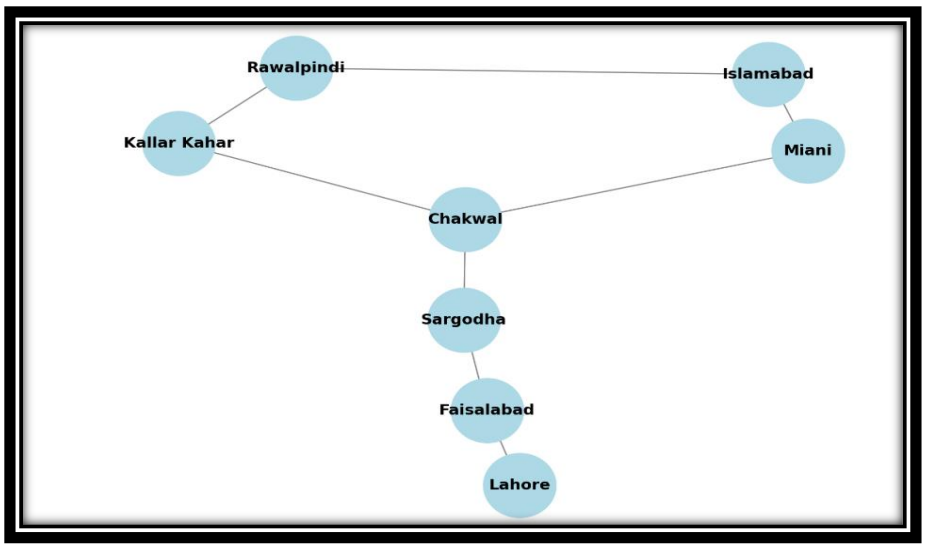
- **Root Node** → Represents the starting state (e.g., the initial city in the pathfinder game).
- **Branches (Edges)** → Connections between states (e.g., roads between cities).
- **Child Nodes** → Possible states that can be reached from the current state.
- **Goal Node** → The desired final state (e.g., reaching Lahore from Islamabad).

How It Works:

- The tree starts from the root node (initial state).
- Each node expands into possible next states, forming branches.
- The search continues until it reaches the goal state.

2) Lab Activity

Path Finder Game:



Code:

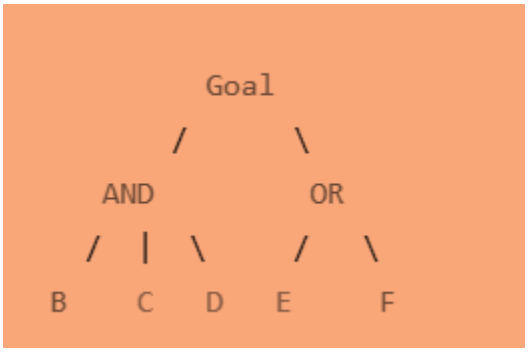
```python
# Define the cities and their direct connections
cities = {
    "Islamabad": ["Rawalpindi", "Miani"],
    "Rawalpindi": ["Islamabad", "Kallar Kahar"],
    "Miani": ["Islamabad", "Chakwal"],
    "Kallar Kahar": ["Rawalpindi", "Chakwal"],
    "Chakwal": ["Miani", "Sargodha"],
    "Sargodha": ["Chakwal", "Faisalabad"],
    "Faisalabad": ["Sargodha", "Lahore"],
    "Lahore": []
}

# Start the game
current_city = "Islamabad"
goal_city = "Lahore"

print("Welcome to the City Path Finder Game!")
print(f"You are starting in {current_city}. Try to reach {goal_city}.\n")

# Game loop
while current_city != goal_city:
    print(f"You are currently in: {current_city}")
    print(f"Possible cities to move to: {', '.join(cities[current_city])}")

    # Get the user's move
    move = input("Where would you like to move? ").strip()

    # Check if the move is valid
    if move in cities[current_city]:
        current_city = move
        print(f"Moving to {current_city}...\n")
    else:
        print("Invalid move! You can only move to cities directly connected to your current city.\n")

# End of game
print(f"Congratulations! You reached {goal_city}!")
```

AND & OR Tree

An AND-OR Tree is a decision-making structure used in problem-solving, especially in AI, to handle multiple possible solutions and conditional dependencies.

AND-OR Tree Combination

- Some problems have both AND and OR conditions.
- Example: A game where you need either a key OR a password to open a door, AND you must also defeat a guard.



Use Case:

- Completing a mission where some tasks are mandatory (AND) while others offer multiple choices (OR).

Make Coffee Maker with AND and OR Trees

```python
while True:
    print("\nWelcome to the Coffee Maker!")
    choice = input("Choose: 1. Brew Coffee  2. Instant Coffee  3. Exit\n")

    if choice == "3":
        print("\nGoodbye!")
        break

    if choice == "1":  # AND Condition
        grind = input("Grind beans? (yes/no): ")
        boil = input("Boil water? (yes/no): ")
        if grind == "yes" and boil == "yes":
            print("\nCoffee brewed successfully!")
        else:
            print("\nBrewing failed! You must do both steps.")

    if choice == "2":  # OR Condition
        instant = input("Choose: 1. Use Coffee Pod  2. Use Instant Powder\n")
        print("\nCoffee ready!")
```

3) Graded Lab Tasks (Allotted Time 1.5 Hours)

1.

Given a **3x3 sliding puzzle Game** with numbers **1 to 8** and a blank tile (**0**), the goal is to rearrange the tiles to match a predefined **goal state** by moving the blank tile in valid direction

2.

Create Path Finder game in which destination from Murree to Karachi.

Hint:

```
"Murree": ["Islamabad"],
    "Islamabad": ["Murree", "Rawalpindi"],
    "Rawalpindi": ["Islamabad", "Kallar Kahar"],
    "Kallar Kahar": ["Rawalpindi", "Chakwal"],
    "Chakwal": ["Kallar Kahar", "Sargodha"],
    "Sargodha": ["Chakwal", "Faisalabad"],
    "Faisalabad": ["Sargodha", "Lahore"],
    "Lahore": ["Faisalabad", "Multan"],
    "Multan": ["Lahore", "Sukkur"],
    "Sukkur": ["Multan", "Hyderabad"],
    "Hyderabad": ["Sukkur", "Karachi"],
    "Karachi": []
```

3.

Python code simulating **AND Gate (Office Door Access)** and **OR Gate (Street Lights Control)** with real-world scenarios